Stress harness — testing beyond the official simulator

`tools/stress_harness.py` (branch `stress_harness`)

Supersedes and merges two earlier tools: `tools/sim_harness.py` (branch `kwongweng_realism_harness` — paraphrase / decoy) and `tools/dual_track_harness.py` (branch `dual_tracking` — browsing-disclosure gating + routing evaluation).

Why

`evaluator/local_evaluator.py` is a fully-cooperative, templated, deterministic customer: it always answers, discloses constraints copied **verbatim** from the target's own metadata, drains every undisclosed constraint on `ask_attribute="other"`, and its "intent override" replaces one target-derived value with another (`behavior_for()` draws both from the same card — never a real retraction). It is a faithful measure of retrieval / ranking / turn efficiency against a customer who has no difficulty to handle. It cannot measure:

- Pillar I routing intelligence (the sim ignores the agent's prose *and* is scenario-agnostic after turn 1, so `FixedPolicy("other")` is unbeatable)
- Pillar II proactive guidance / genuine override
- Pillar III adaptation (nothing to adapt to)

The 800 private sessions "may paraphrase." This harness quantifies what that costs, and — for each failure — says whether **retrieval** or **ranking** broke.

What it does

Drives the **unmodified** `Agent` through a faithful copy of `evaluate()`'s session loop with composable customer stressors:

stressor	behaviour
`paraphrase:light`	same constraints, verbatim tokens kept, only the carrier sentence reworded (`"For that, what matters is: X"` -> `"It should be X."`)
`paraphrase:medium`	the constraint itself reworded via ~5 patterns (`"color: blue"` -> `"in blue"`; `"100% Cotton"` -> `"all cotton"`; `"buckle closure"` -> `"a buckle fastening"`). Rule-based, deterministic per session.
`paraphrase:heavy`	`medium` + **broad synonym substitution** (`leather` -> `cowhide` / `tanned-hide build`, `waterproof` -> `water-repellent`, `hiking` -> `trekking`, `stainless steel` -> `surgical-grade metal`) + clause shuffle/fusion + spoken filler. ~70 synonym entries, each with at least one option that drops the key token, so it erodes FTS5 recall and not only the span signal. Still rule-based — an offline LLM rewriter is the real "heavy".
`browse-gated`	the **browsing** customer discloses a constraint only when asked a *pointed* question whose `classify_constraint` bucket matches — never on the broad `"other"`. Makes Buying/Browsing routing load-bearing.
`decoy`	`intent_override` sessions where the pre-override preference is a **genuine decoy** — a colour/material value the target does not have and whose token is absent from its text. The override becomes a real retraction.

Stressors compose: `--customer paraphrase:medium+browse-gated` is a vague browser who also rewords everything.

Per scenario it also reports a **retrieval diagnostic** — `never_retrieved` (target never entered the 300-pool), `pool_rank>100`, `median_pool_rank`, `ranked_out` (in the pool, never surfaced) — computed by recomputing `retrieve()` from the accumulated state. `--targets generic` restricts to targets whose disclosed constraint spans are *all* high-frequency in the catalog (>=400 products each), so BM25 cannot separate the target and it lands deep in the pool.

```
python3 tools/stress_harness.py --verify                      # delta 0 vs local_evaluator
python3 tools/stress_harness.py --all                         # the stressor matrix
python3 tools/stress_harness.py --customer paraphrase:medium+browse-gated
python3 tools/stress_harness.py --customer browse-gated --configs router_off,router_on
python3 tools/stress_harness.py --customer browse-gated --misroute-matrix
python3 tools/stress_harness.py --all --targets generic
```

`--verify` asserts the un-stressed path reproduces whatever agent it is given (delta 3.6e-07 for the branch default). Reported numbers are a **robustness probe**, not the official score.

Results — branch default agent (`use_router=True`), PUBLIC 200

Official (this agent) = 0.91768.

customer	hit@10	MRR	score	Δ	tok_cov
official	1.000	0.868	0.91768	—	0.792
paraphrase:light	0.990	0.793	**0.88689**	-0.031	0.806
paraphrase:medium	0.995	0.775	**0.88216**	-0.036	0.709
paraphrase:heavy	0.970	0.752	**0.85577**	**-0.062**	0.660
browse-gated	0.980	0.793	**0.87745**	-0.040	0.678
paraphrase:medium + browse-gated	0.935	0.728	**0.82510**	-0.093	0.616
paraphrase:heavy + browse-gated	0.930	0.669	**0.80071**	**-0.117**	0.573
decoy	1.000	0.876	0.91937	+0.002	0.797

`heavy` roughly doubles `medium`'s drop (-0.062 vs -0.036) and takes `tok_cov` down to 0.66 — it is eroding real constraint tokens, not just phrasing. `heavy + browse-gated` = **0.80** — the closest single number to a private simulator that both paraphrases and has non-cooperative browsers.

decoy: 26/30 override sessions got a real decoy; still handled (+0.002) — the `focused_text` route + `_facet_conflicts` recover. Override machinery is not dead weight, it is private-set insurance the official metric hides.

Retrieval vs ranking — `heavy + browse-gated`, per scenario

scenario	score	never_retrieved	median pool_rank	ranked_out
buying	0.842	**2 / 80**	9	1 / 80
browsing	0.721	**10 / 80**	13	1 / 80
intent_override	0.877	0 / 30	2	0 / 30
boundary	0.878	0 / 10	5	0 / 10

(`medium + browse-gated` is the same shape — browsing `never_retrieved` 9/80, buying 1/80.)

This is the finding. Under a realistic browsing customer who also paraphrases, **~12% of browsing targets never enter the retrieval pool** and the median pool rank doubles (7 -> 13). Buyers are almost unaffected (1/80) — they front-load a hard constraint in the opening, so their query is strong even paraphrased. The retrieval weakness is

track-asymmetric and concentrated on the browsing side, because a gated + paraphrased browser gives 1–2 vague, non-verbatim terms and the bag-of-words query has nothing to bite on.

Generic-target subset (21 of 200 — all constraints high-frequency)

customer	hit@10	MRR	score	Δ
official	1.000	0.636	0.82122	—
paraphrase:medium	0.952	0.434	0.73395	-0.087
paraphrase:heavy	0.905	0.386	0.68622	-0.135
paraphrase:medium + browse-gated	0.810	0.432	0.64306	-0.178
paraphrase:heavy + browse-gated	0.762	0.348	**0.58621**	**-0.235**

Even fully cooperative, generic targets rank far worse (MRR 0.87 -> 0.64, `intent_override` median pool_rank 40). This subset is where the reranker's verbatim-span signal has the least to work with, and it compounds hard with paraphrase — `heavy + browse-gated` here is **0.59**, the worst plausible cell.

Routing under the harness (unchanged from branch `dual_tracking`)

`--customer browse-gated --configs router_off,router_on`:

	router_off	router_on	Δ
overall	0.7308	0.8775	**+0.147**
browsing hit / MRR	0.59 / 0.24	0.95 / 0.67	
buying	1.00 / 0.90	1.00 / 0.90	identical

Misroute 2×2 (`--misroute-matrix`): browser-as-buyer 0.59 / 0.24 vs buyer-as-browser 1.00 / 0.83 — ~10× asymmetric. See `docs/team/dual_track_routing.md`.

Implications for the build

1. **Paraphrase is the largest untested exposure.** `light`/`medium` -0.03/-0.04, `heavy` -0.06, `heavy + browse-gated` **-0.12** (score 0.80), and on the hard-to-retrieve subset `heavy + browse-gated` is **-0.24** (score 0.59). The loss is not missed constraints — `constraint_spans()` fails to yield a clean fragment for the verbatim-substring reranker (the dominant S6 signal), and at `heavy` the tokens themselves change so FTS5 recall also erodes. If the private simulator paraphrases *and* its browsers are non-cooperative, the real score is **~0.80**, and **~0.59** on the tail. (`heavy` is still rule-based; an offline LLM rewriter would likely be worse again.)
1. **There IS a retrieval gap, and it is on the browsing track** — `never_retrieved` 9/80 for gated+paraphrased browsers vs 1/80 for buyers. This is the first hard evidence for a *track-routed retrieval*, and it points the opposite way to the intuition "narrow the buyer's pool": buyers are the healthy case; **the browsing route is the one that needs help** — query expansion (catalog co-occurrence / synonyms), or a category-only fallback when disclosure is too sparse to form a query. Pool *size* is not the lever (`rerank.py:depth=300` exists because targets already sit deep in BM25 order; shrinking it drops them).
1. **Robustness fix worth pursuing:** loosen the reranker's constraint match from exact substring to token-set overlap. A `bge-small` **bi-encoder cosine** was added as an S6 term and tested here (`docs/team/dense_rerank.md`): it helps only on the fully-degenerate tail (`--targets generic` +0.021) and is a wash to -0.003 on the realistic full set, because the retrieved pool is already category-homogeneous so the embedding discriminates attributes weakly. The real semantic lever is a **cross-encoder** (branch `semantic-rerank-experiment`), never tested under paraphrase.

1. **Write-up:** "Innovation & Problem Insight" (20% of judging) is scored on how clearly the challenge is framed. This harness + these numbers are that section: *what the public simulator can't measure, and how we tested beyond it.*

Extending

`StressCustomer` composes flags in `_disclose` / `_render` / `opening` / `__init__`. To add a stressor, add a flag and one hook:

- `PartialDisclosure` — disclose only 1 of N constraints even when asked broadly; tests whether the agent digs.
- `NoisyCustomer` — 40% "I'm not sure"; occasional contradiction with no "actually" cue. Tests state-machine robustness.
- An **LLM paraphraser** (offline model or API, disclosed as an eval-time tool, never part of the agent) would replace `_reword_one` for a `heavy+` level. Wire it as a **cache-keyed lazy rewrite**: the first time a constraint string is needed at a given level, call the model and store the result under `(constraint, level, prompt-version)`; every run after the first reads from disk. Same pattern as the `bge-small` embedding cache the tree used to carry (`tools/build_embeddings.py`, removed in change 20) — it keeps `--verify` deterministic and the dozens of A/B passes cheap. The rewriter must see only the constraint text (never the target id) and its output is validated (meaning preserved, nothing dropped) with the rule-based `heavy` as the fallback.